Le Fin

Project: P05 — THIS IS JEOPARDY!!!!!!!

Team: Pomegranates

Roster: Jason Chan (PM), Artemis Lee, Ethan Cheung

--

PROJECT DESCRIPTION:

Users will be able to play a game of Jeopardy with friends. They may use pre-generated boards created from our database of questions, or they may make their own customizable gameboard. Upon playing a game, the user will reveal clues and take keyboard inputs from the players. A player will gain/lose points depending on if they give the correct answer. Obviously, whoever has the most points wins. Have fun!

--

PROGRAM COMPONENTS + EXPLANATION:

Python files

- *__init__.py*: The main file; serves app
- *data_setup.py*: Handles parsing of CSV file and creation of Sqlite3 database tables
- *data.py*: Handles the data in the Sqlite3 grocery database and the user data (all stored in *data.db*)

Templates

- *login.html*: The user can login to their account or register if they don't have one. Redirects to the homepage.
- *register.html*: The user will be able to register. Redirects to the homepage.
- *home.html*: The homepage, which will allow users to start a new game with created boards/pre-generated boards, or start creating/editing a custom board.
- create_new.html: When creating a new custom board, the user will have to give it a unique title before adding in their clues.

- *create.html*: The user can add categories and # of clues, then for each clue, insert $, question, and answer. Each clue will be saved as a string by default, so users can save unfinished boards and complete them later.
- edit_slot: The user can edit a specific slot of their own board.
- *edit_board.html*: The user can go back to their own custom boards and edit them as desired. Redirects to the create page.
- *game.html*: Takes user input for the game, allowing them to choose what questions to answer (choosing a question will redirect a user to buzzer.html), and shows score lists.
- *buzzer.html*: Allows users to "buzz in", taking user input times to determine who is allowed to answer first, and giving contestants a prompt that is compared to the actual answer to see if they're correct. Note that all players will be on the same computer, with each player on their own designated key to press.
- *profile.html*: Allows users to see their username, description, points, current game, and high score
- *edit_profile.html*: Edit username or description.

JavaScript files

- *buzzer.js*: Handles the buzzer inputs during a game.

CSS files/FEF

- We will be using Tailwind because it is awesome. We will not be using a CSS file.

----------

DATABASE ORGANIZATION:

We will be using SQLite3 to organize our data; more specifically the user information, their saved boards, and current game stats. We will not be using any databases from external sources.

Tables:

users

Variable Type	Variable Name	Variable Attribute(s)
STRING	username	PK NOT NULL
STRING	password	NOT NULL
STRING	bio	NOT NULL
INT	total_points	NOT NULL
INT	wins	NOT NULL
INT	runnerups	NOT NULL
INT	losses	NOT NULL

board

Variable Type	Variable Name	Variable Attribute(s)
STRING	title	PK NOT NULL
INT	row	NOT NULL
INT	column	NOT NULL
STRING	question_category	NOT NULL
STRING	answer	
STRING	wrong1	
STRING	wrong2	
STRING	wrong3	
INT	point_value	
INT	chosen	NOT NULL

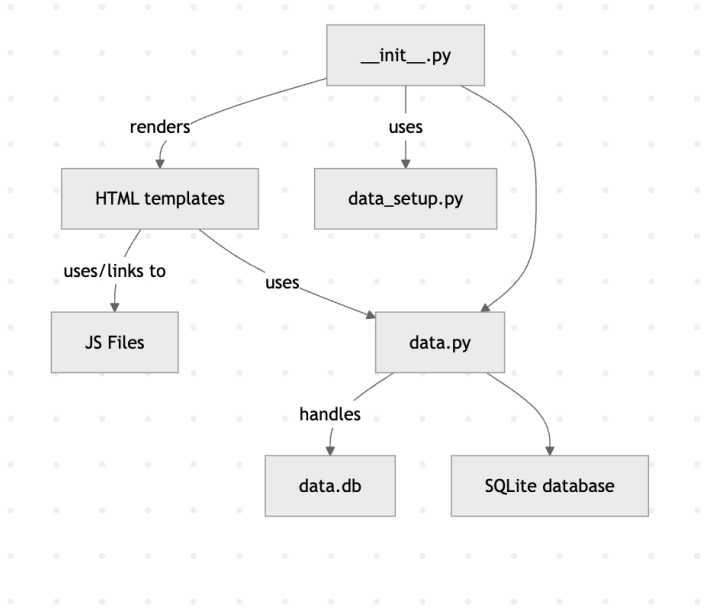
game

Variable Type	Variable Name	Variable Attribute(s)
STRING	player1	NOT NULL
STRING	player2	NOT NULL
STRING	player3	NOT NULL
INT	points1	NOT NULL
INT	points2	NOT NULL
INT	points3	NOT NULL

APIs:

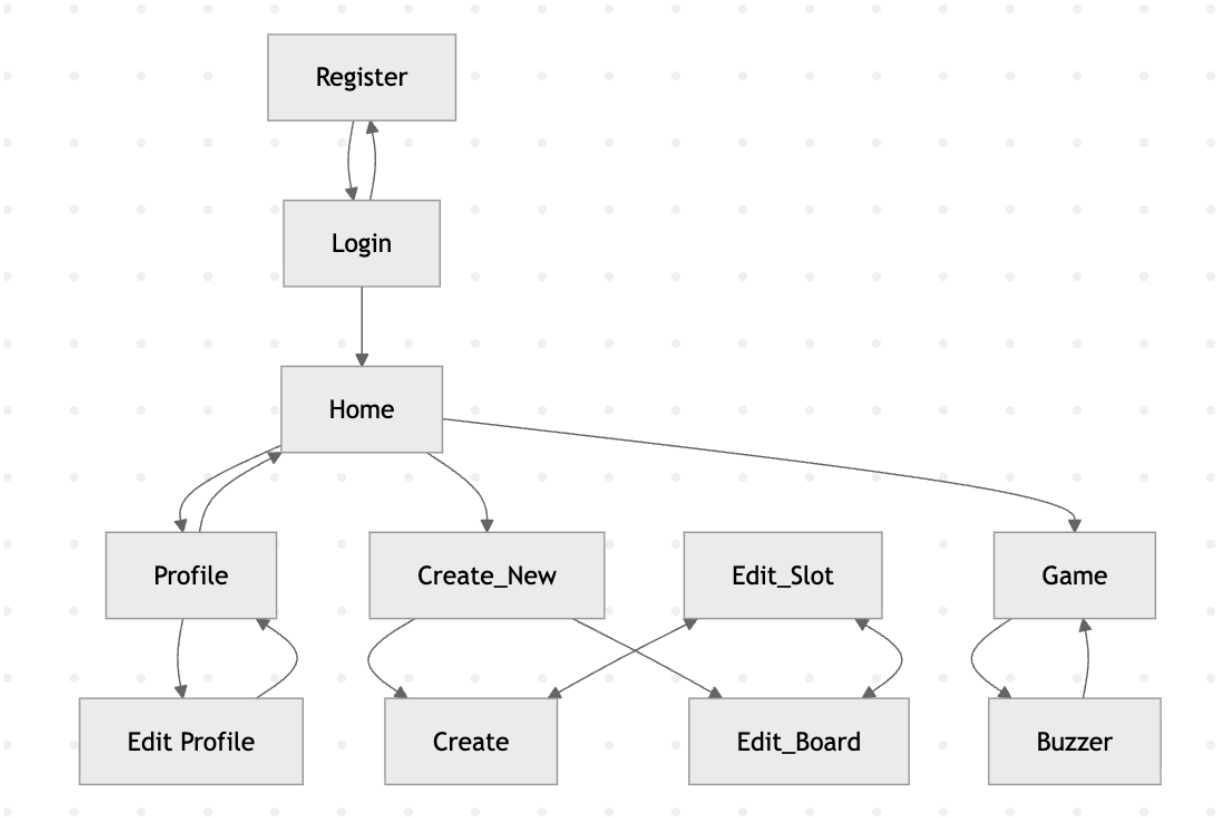
Open Trivia API: Provides a whole bunch of trivia questions as requested. Responsive and easy to filter, but only outputs 50 questions at a time. No sign up required, just provides a link.

COMPONENT MAP:



--

FRONT END DIAGRAM:



TIMELINE:

WEEK 1:

- Get basic flask app and HTML template structure working
- Login/Register
- data_setup.py completed or close to complete

WEEK 2:

- Functions for data.py completed
- OpenTDB API implementation
- Users should be able to create/edit game boards

WEEK 3:

- Playing the game should be functional

    - Displaying clues
    - Buzzer inputs
    - Keeping track of stats throughout the game
- Any stretch goals we may have time for
    - Being able to play other users' custom boards
    - Leaderboards for various stats (wins/losses, total points, correct responses, etc.

TASK DELEGATION (REALLY SUBJECT TO CHANGE! DEVOS ARE NOT FULLY RESTRICTED TO ASSIGNED TASKS!):

Task	**Devo(s)**	**Deadline**
Html stuff (page rendering)	DEC	05-18 @ 11:06
Tailwind stuff (styling)	DEC	05-30 @ 23:59
Database handling (making and applying data_setup.py and data.py)	DJC	05-21 @ 11:06
API stuff (implementing OpenTDB API)	DJC	05-21 @ 11:06
Python stuff (making flask work)	DAL	05-15 @ 23:59
Javascript stuff (making buzzer work)	DAL	05-26 @ 11:06
Devlog	EVERYONE	06-01 @ 08:00